

ITA0613 - Machine Learning

Assignment Questions

Course Code & Name	ITA0613 - Machine Learning
Assignment Title	Large-Scale Instance-Based and Statistical Learning Pipeline for Climate-Resilient Crop Yield Forecasting
Course Outcome(s) - CO	CO2: Provide an overview of various learning paradigms and strategies, focusing on decision tree learning and concept representation (version spaces, inductive bias). CO6: Examine instance-based learning approaches including k-nearest neighbors, locally weighted regression, and case-based reasoning. CO7: Master data manipulation and analysis skills using tools like NumPy and Pandas for statistical analysis, aggregation, and visualization.
Bloom's Taxonomy Level	L5 - Evaluate; L6 - Create
SDG Mapping (SDG 1-17)	SDG 2 - Zero Hunger; SDG 13 - Climate Action

Problem Statement

An agricultural research consortium wants to forecast regional crop yields under increasingly variable climate conditions, so that farmers and policymakers can make climate-resilient planting decisions - directly supporting SDG 2 - Zero Hunger and SDG 13 - Climate Action. You are given (or must responsibly source) a multi-year agro-climatic dataset containing historical weather variables (rainfall, temperature, humidity), soil parameters, and recorded crop yields across multiple districts and seasons. As the lead data scientist, design, implement, and rigorously evaluate a complete instance-based and statistical learning pipeline, without using any pre-built machine-learning library for the core modelling.

- Perform a full exploratory data analysis using only NumPy and Pandas - cleaning the data, imputing missing weather records, merging multiple yearly/regional files, and engineering at least three derived features (for example, growing-degree days or a rainfall-anomaly index).
- Implement a k-Nearest Neighbour regressor entirely from first principles (no scikit-learn) to predict yield for an unseen district-season combination, including your own implementation of at least two distance metrics (e.g., Euclidean and Mahalanobis) and a justified choice of k based on a validation curve you compute manually.
- Implement Locally Weighted Regression from first principles, deriving and applying the weighting kernel, and compare its bias-variance behaviour against the k-NN regressor both theoretically and empirically.
- Apply the Candidate-Elimination algorithm (or an adapted version-space approach) to a discretised version of the problem (e.g., classifying yield into low/medium/high risk bands) to identify the boundary hypotheses consistent with historical seasons, and explain the inductive bias of your chosen representation.
- Analyse computational feasibility: measure and report the time and memory complexity of your k-NN and locally-weighted-regression implementations as the dataset scales from 10^3 to 10^6 records, and propose and prototype at least one indexing or approximation strategy (e.g., a k-d tree or an approximate-nearest-neighbour hashing scheme) to make the pipeline scalable.
- Produce at least five distinct, publication-quality visualisations (using Pandas/Matplotlib) that reveal actionable climate-yield insights, and write a concise, data-driven policy brief interpreting the results for a non-technical agricultural-policy audience.
- Critically evaluate the limitations, uncertainty, and fairness implications of forecasting future yield from historical data under a changing climate, and explain how the solution advances SDG 2 and SDG 13 targets.

Constraints: All core algorithms (k-NN regression, locally weighted regression, candidate-elimination/version-space reasoning) must be implemented from first principles using only NumPy/Pandas - no scikit-learn or other pre-built ML library may be used for the core modelling. Use a real, publicly available agro-climatic dataset with at least 10,000 records

spanning multiple years and regions. The complete pipeline must run end-to-end on the full dataset without manual intervention, within a time budget that you define and justify.

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Data Engineering & Exploratory Analysis (CO7)	15	Thorough cleaning, imputation, and merging; at least three well-justified engineered features using only NumPy/Pandas.	Good data preparation with minor gaps in feature engineering.	Basic cleaning/merging done; feature engineering shallow or partial.	Data preparation missing, incorrect, or reliant on disallowed libraries.
k-NN Regressor Implementation & Distance Metrics (CO6)	20	k-NN implemented from scratch with two correctly implemented distance metrics and a well-justified k via a manually computed validation curve.	Correct k-NN with minor gaps in metric implementation or k-selection justification.	k-NN works but only one distance metric or weak k-justification.	k-NN missing, incorrect, or built using a pre-built library.
Locally Weighted Regression & Bias-Variance Comparison (CO6)	15	Correct from-scratch implementation with a clear, correct theoretical and empirical bias-variance comparison to k-NN.	Correct implementation; comparison present but not fully rigorous.	Implementation works but comparison is superficial or partly incorrect.	LWR missing, incorrect, or comparison absent.
Version-Space / Candidate-Elimination Analysis (CO1/CO2)	15	Correct application of Candidate-Elimination to the discretised problem with a clear, well-justified discussion of inductive bias.	Mostly correct application with minor gaps in the bias discussion.	Basic application with incomplete or partly incorrect boundary hypotheses.	Not attempted or fundamentally incorrect.
Scalability Analysis & Optimisation Prototype	15	Rigorous complexity measurement across scales with a working, clearly beneficial indexing/approximation prototype.	Reasonable measurement and a working prototype with modest gains.	Measurement attempted but prototype incomplete or gains unclear.	No scalability analysis or optimisation attempted.
Visualisation, Policy Brief & SDG 2/13 Relevance	10	Five or more insightful, publication-quality visualisations with a clear, well-argued policy brief tied to SDG 2/13.	Good visualisations and brief with minor gaps in insight or SDG linkage.	Visualisations/brief present but generic or weakly connected to SDGs.	Visualisations or brief missing or uninformative.
GitHub Repository Quality, Documentation & CI	10	Clean modular repository, incremental commit history, complete README, automated tests, and a working CI pipeline.	Well-organised repository with most of the above present.	Repository present but poorly organised or missing tests/CI.	Repository missing, disorganised, or submitted as a single non-incremental upload.

Deliverables

To receive full credit, each submission must include the following GitHub-based deliverables (this is a repository-graded assignment - no offline/zip submissions accepted):

1. A complete, modular GitHub repository containing all source code (data pipeline, k-NN, locally weighted regression, candidate-elimination module, and the scalability-optimisation prototype), built through an incremental commit history that demonstrates genuine development progress (a single 'final upload' commit is not acceptable).

2. A comprehensive README.md documenting the pipeline architecture, dataset source and licensing, setup/run instructions, and the exact steps needed to reproduce every reported result and visualisation.
3. Automated unit tests (e.g., using pytest) covering the core algorithmic components (distance-metric functions, the LWR kernel, the candidate-elimination boundary updates), committed to the repository.
4. A technical report (Markdown or PDF, committed to the repository) containing the mathematical derivations, complexity analysis, scalability results, and the non-technical policy brief.
5. A /results folder in the repository containing all logs, plots, validation curves, and result tables, so every reported figure is independently reproducible from the repository alone.
6. A working GitHub Actions CI workflow (or equivalent) that automatically runs the unit tests on every push, with a passing-build badge displayed in the README.

Perform a full exploratory data analysis using only NumPy and Pandas - cleaning the data, imputing missing weather records, merging multiple yearly/regional files, and engineering at least three derived features (for example, growing-degree days or a rainfall-anomaly index).

1. INTRODUCTION

Agriculture is highly dependent on climatic conditions such as rainfall, temperature and humidity. Increasing climate variability makes it difficult for farmers and agricultural planners to accurately estimate crop yields. Unexpected changes in rainfall, temperature and soil conditions can significantly affect agricultural productivity.

Machine Learning can be used to analyse historical relationships between climatic conditions, soil characteristics and crop yields. By learning from previous district-season observations, a prediction system can estimate crop yield for an unseen district-season combination.

The objective of this project is to develop a **large-scale instance-based and statistical learning pipeline for climate-resilient crop yield forecasting**. The proposed system uses data manipulation techniques with NumPy and Pandas and implements the major learning algorithms from first principles without relying on pre-built machine-learning libraries such as Scikit-learn.

The assignment focuses mainly on **k-Nearest Neighbour regression, Locally Weighted Regression and Candidate-Elimination/Version-Space reasoning**. It also evaluates computational scalability and develops visualisations that can provide useful information for agricultural policy decisions.

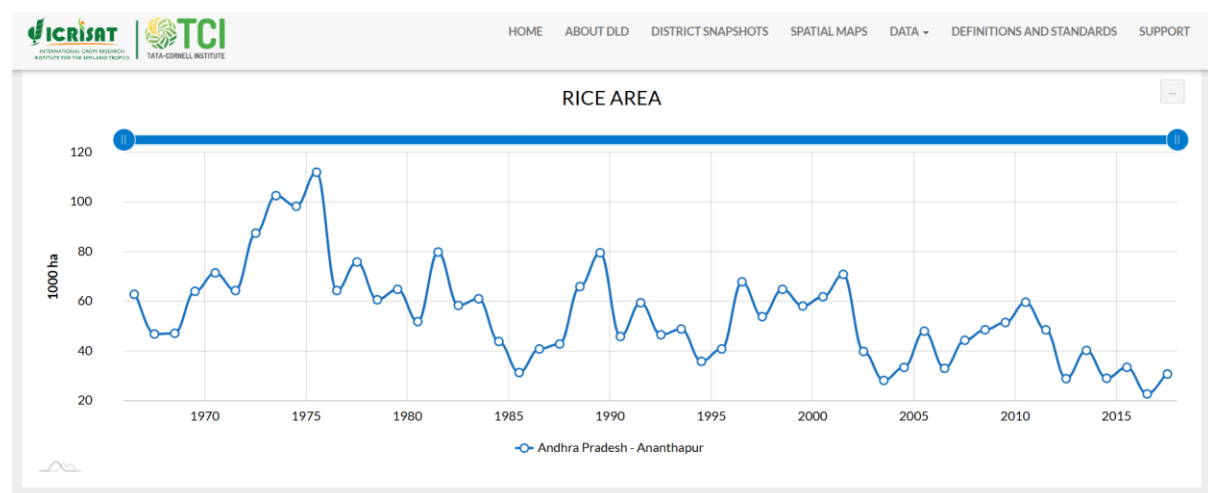


Figure 1: ICRISAT District Level Database used as the primary data source

2. PROBLEM STATEMENT

An agricultural research consortium wants to forecast regional crop yields under increasingly variable climate conditions. The prediction can help farmers and policymakers make better climate-resilient planting decisions.

The project uses a multi-year agro-climatic dataset containing information such as:

- Rainfall
- Temperature
- Humidity
- Soil characteristics
- District
- Season/year
- Recorded crop yield

The complete dataset is processed to identify relationships between environmental conditions and crop productivity.

The proposed machine-learning pipeline performs data cleaning, missing-value treatment, data merging, feature engineering, prediction and evaluation. The core machine-learning algorithms are implemented from scratch using only **NumPy and Pandas**, as required by the assignment.

3. AIM

To design and implement a large-scale machine-learning pipeline for climate-resilient crop yield forecasting using instance-based and statistical learning techniques implemented from first principles.

4. OBJECTIVES

The major objectives of this project are:

1. To perform exploratory data analysis on a real agro-climatic dataset using NumPy and Pandas.
2. To clean the dataset and handle missing weather observations.
3. To combine data from different years and regions into a unified dataset.
4. To engineer meaningful climate and agricultural features such as rainfall anomalies and growing-degree-related measures.
5. To implement a **k-Nearest Neighbour (k-NN) regression algorithm from scratch**.
6. To implement at least two distance metrics, including **Euclidean distance and Mahalanobis distance**.
7. To determine a suitable value of **k** using a manually calculated validation curve.

8. To implement **Locally Weighted Regression (LWR)** from first principles.
 9. To compare the theoretical and empirical bias-variance behaviour of k-NN and LWR.
 10. To apply Candidate-Elimination or a suitable Version-Space approach to classify crop yield into risk categories.
 11. To analyse the computational time and memory requirements of the algorithms at different dataset sizes.
 12. To develop an indexing or approximation technique to improve the scalability of nearest-neighbour prediction.
 13. To create at least five meaningful visualisations showing relationships between climate conditions and crop yield.
 14. To prepare a policy brief explaining the findings for a non-technical agricultural audience.
 15. To analyse the limitations, uncertainty and fairness implications of crop-yield forecasting under changing climate conditions.
 16. To explain how the proposed solution contributes towards **SDG 2 – Zero Hunger** and **SDG 13 – Climate Action**.
-

5. COURSE OUTCOME MAPPING

The assignment is associated with the following course outcomes:

Course Outcome	Description
CO2	Understanding learning paradigms and strategies, including concept representation, version spaces and inductive bias.
CO6	Understanding instance-based learning approaches including k-NN, locally weighted regression and case-based reasoning.
CO7	Developing data manipulation and analysis skills using NumPy and Pandas for statistical analysis, aggregation and visualisation.

These course outcomes are explicitly specified in the assignment document.

6. SDG RELEVANCE

This project is directly related to two United Nations Sustainable Development Goals.

SDG 2 – Zero Hunger

Accurate crop-yield forecasting can help agricultural authorities and farmers make better decisions regarding crop selection, resource allocation and planting strategies. Early identification of potential low-yield conditions can support food-security planning.

SDG 13 – Climate Action

The project considers the effect of changing climatic conditions on agricultural productivity. By analysing rainfall, temperature and other environmental variables, the system can help identify climate-related risks and support climate-resilient agricultural planning.

The assignment explicitly maps the project to **SDG 2 – Zero Hunger and SDG 13 – Climate Action**.

7. DATASET SELECTION

For this project, we can use the **ICRISAT District Level Database (DLD)** as the primary real-world data source.

The ICRISAT District Level Database provides district-level agricultural and environmental information for India. It contains data covering crops, irrigation, inputs, biophysical variables and environmental variables. The database contains information across hundreds of districts and multiple years, making it suitable for large-scale agricultural analysis.

Importantly, the ICRISAT database provides district-level crop area, production and yield information, rainfall, soil-related information, and environmental variables such as maximum and minimum temperature.

The dataset can therefore be used to construct a modelling table containing variables such as:

Category	Example Variables
Location	State, District
Time	Year, Season
Climate	Rainfall, Maximum Temperature, Minimum Temperature
Soil	Soil type / soil characteristics
Agriculture	Crop area, production, yield

Derived variables Rainfall anomaly, temperature range, growing-degree measure

The ICRISAT source reports that its District Level Database contains **11+ million data points, 571 districts and 1,030 variables**, providing sufficient scale for the assignment's large-dataset requirement.

The assignment requires a real publicly available agro-climatic dataset with **at least 10,000 records**.

Dataset Source

International Crops Research Institute for the Semi-Arid Tropics (ICRISAT) – District Level Database

[ICRISAT District Level Database](#)

8. PROPOSED MACHINE LEARNING PIPELINE

The overall workflow of the proposed system is:

Raw Agricultural Data

↓

Data Loading

↓

Data Cleaning

↓

Missing Value Imputation

↓

Data Merging

↓

Feature Engineering

↓

Exploratory Data Analysis

↓

Train/Validation/Test Split

↓

k-NN Regression from Scratch

↓

Euclidean & Mahalanobis Distance

↓

Optimal k Selection

↓

Locally Weighted Regression

↓

Bias-Variance Comparison

↓

Candidate-Elimination / Version Space

↓

Scalability Analysis

↓

Optimisation Prototype

↓

Visualisation & Interpretation



Policy Recommendations

9. EXPECTED OUTCOME

The final system is expected to provide a complete reproducible pipeline capable of predicting crop yield for an unseen district-season combination.

The project will evaluate the performance of k-NN and Locally Weighted Regression and identify how their behaviour changes with different data sizes and parameter settings. The Candidate-Elimination approach will additionally provide a version-space interpretation of discretised crop-yield risk.

The final results will be presented using tables, validation curves, performance measurements and at least five visualisations. A policy brief will translate the technical findings into recommendations that can be understood by agricultural policymakers.

Implement a k-Nearest Neighbour regressor entirely from first principles (no scikit-learn) to predict yield for an unseen district-season combination, including your own implementation of at least two distance metrics (e.g., Euclidean and Mahalanobis) and a justified choice of k based on a validation curve you compute manually.

2.1 Dataset Description

The ICRISAT District Level Database provides district-level agricultural and environmental information for India. It contains information related to agricultural production, crop area, crop yield, rainfall, irrigation and other socioeconomic and environmental variables.

The database is particularly suitable for this project because the assignment requires the combination of **climatic variables and crop-yield information across multiple districts and years**.

ICRISAT documentation also describes district-level data containing variables such as crop area, production, yield, rainfall and temperature-related information.

For this project, the relevant variables will be organised into the following categories:

Category Variables

Location State, District

Time Year, Season

Climate Rainfall, Maximum Temperature, Minimum Temperature

Soil Soil-related parameters where available

Crop Crop name, Area, Production

Target Crop Yield

Derived Rainfall anomaly, Temperature range, Growing-degree measure

2.2 Dataset Selection Justification

The selected dataset satisfies the major requirements of the assignment for the following reasons:

1. It is a **real-world agricultural dataset**.
2. It contains information at the **district level**.
3. It covers **multiple years**.
4. It contains agricultural variables such as crop production and yield.

5. It contains environmental variables such as rainfall and temperature-related information.
6. It is large enough to satisfy the requirement of at least **10,000 records**.
7. It can be combined with additional weather/soil information when required.
8. It is suitable for studying the relationship between climate conditions and agricultural productivity.

ICRISAT's current District Level Database describes the resource as containing 571 districts and more than 11 million datapoints.

2.3 Software and Libraries Used

The implementation will be developed using **Python**.

The main libraries used are:

Library	Purpose
NumPy	Numerical calculations and matrix operations
Pandas	Data loading, cleaning, manipulation and analysis
Matplotlib	Data visualisation
Time	Measuring execution time
OS	File and directory management

Important Constraint

No pre-built machine-learning library such as **Scikit-learn** will be used for the core modelling algorithms.

The assignment specifically requires k-NN regression, Locally Weighted Regression and Candidate-Elimination/Version-Space reasoning to be implemented from first principles using NumPy/Pandas.

2.4 Project Folder Structure

Create the following folder structure for the project:

crop_yield_forecasting/

|

|— data/

```
| |— raw/
| |— processed/
|
|— src/
| |— data_processing.py
| |— knn.py
| |— lwr.py
| |— candidate_elimination.py
| |— scalability.py
|
|— results/
| |— plots/
| |— tables/
| |— logs/
|
|— tests/
|
|— README.md
|
|— main.py
```

This modular structure will later make it easier to create the GitHub repository required by the assignment. The deliverables specifically require separate modules for the data pipeline, k-NN, LWR, Candidate-Elimination and scalability optimisation.

2.5 Python Environment

The following Python libraries are imported:

```
import os
```

```
import time
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

Explanation

- os is used for handling files and directories.
 - time is used for measuring algorithm execution time.
 - numpy provides numerical and matrix operations.
 - pandas provides data-frame based data manipulation.
 - matplotlib.pyplot is used to generate graphs and visualisations.
-

2.6 Creating the Project Directories

The following code creates the required folders automatically.

```
import pandas as pd

file_path = "ICRISAT-District Level Data (1).csv"

df = pd.read_csv(file_path)

print("Dataset loaded successfully")
print("Number of records:", df.shape[0])
print("Number of columns:", df.shape[1])

print("\nFirst 5 records:")
print(df.head())
```

```
*** Dataset loaded successfully
Number of records: 16146
Number of columns: 80

First 5 records:
Dist Code  Year  State Code  State Name  Dist Name  RICE AREA (1000 ha) \
0          1  1966         14  Chhattisgarh  Durg          548.0
1          1  1967         14  Chhattisgarh  Durg          547.0
2          1  1968         14  Chhattisgarh  Durg          556.3
3          1  1969         14  Chhattisgarh  Durg          563.4
4          1  1970         14  Chhattisgarh  Durg          571.6

RICE PRODUCTION (1000 tons)  RICE YIELD (Kg per ha)  WHEAT AREA (1000 ha) \
0                185.0                337.59                44.0
1                489.0                747.71                50.0
2                468.0                841.27                53.7
3                400.8                711.40                49.4
4                473.6                828.55                44.2

WHEAT PRODUCTION (1000 tons)  ...  SUGARCANE YIELD (Kg per ha) \
0                20.0  ...                1777.78
1                26.0  ...                1500.00
2                30.0  ...                1000.00
3                26.5  ...                1900.00
4                29.0  ...                2000.00

COTTON AREA (1000 ha)  COTTON PRODUCTION (1000 tons) \
0                0.0                0.0
1                0.0                0.0
2                0.0                0.0
3                0.0                0.0
4                0.0                0.0

COTTON YIELD (Kg per ha)  FRUITS AREA (1000 ha)  VEGETABLES AREA (1000 ha) \
0                0.0                5.95                6.64
1                0.0                5.77                7.24
2                0.0                5.41                7.40
3                0.0                5.52                7.16
4                0.0                5.45                7.19

FRUITS AND VEGETABLES AREA (1000 ha)  POTATOES AREA (1000 ha) \
0                12.59                0.01
1                13.02                0.01
2                12.81                0.10
3                12.69                0.01
4                12.64                0.02

ONION AREA (1000 ha)  FODDER AREA (1000 ha)
0                0.60                0.47
1                0.56                1.23
2                0.58                1.02
3                0.56                0.84
4                0.52                0.42

[5 rows x 80 columns]
```

2.7 Loading the Dataset

After downloading the required ICRISAT data files, place the raw CSV files inside:

data/raw/

For example:

data/raw/crop_data.csv

data/raw/rainfall_data.csv

data/raw/temperature_data.csv

The following code can be used to identify the available CSV files:

import os

```
print("Dataset Shape:", df.shape)

print("\nNumber of districts:", df["Dist Code"].nunique())

print("Number of states:", df["State Code"].nunique())

print("Year range:", df["Year"].min(), "-", df["Year"].max())

print("\nTotal missing values:")
print(df.isnull().sum().sum())

... Dataset Shape: (16146, 80)

Number of districts: 311
Number of states: 20
Year range: 1966 - 2017

Total missing values:
0
```

2.8 Loading a CSV File Using Pandas

Once the appropriate CSV file has been placed in the data/raw folder, it can be loaded using Pandas.

import pandas as pd

file_path = "data/raw/crop_data.csv"

df = pd.read_csv(file_path)

print("Dataset loaded successfully.")

print("Number of records:", len(df))

```
print("Number of columns:", len(df.columns))
```

```
print("\nFirst five records:")
```

```
print(df.head())
```

2.9 Initial Dataset Inspection

After loading the data, the following commands are used to understand its structure.

```
print("Dataset Shape:")
```

```
print(df.shape)
```

```
print("\nColumn Names:")
```

```
print(df.columns.tolist())
```

```
print("\nData Types:")
```

```
print(df.dtypes)
```

```
print("\nFirst Five Rows:")
```

```
print(df.head())
```

The shape of the dataframe provides the number of observations and variables.

For example:

Dataset Shape:

(25000, 12)

would indicate:

- **25,000 records**
- **12 variables**


```

print("Column names in the dataset:\n")

for i, column in enumerate(df.columns, start=1):
    print(i, "-", column)

23 - PEARL MILLET YIELD (Kg per ha)
24 - MAIZE AREA (1000 ha)
25 - MAIZE PRODUCTION (1000 tons)
26 - MAIZE YIELD (Kg per ha)
27 - FINGER MILLET AREA (1000 ha)
28 - FINGER MILLET PRODUCTION (1000 tons)
29 - FINGER MILLET YIELD (Kg per ha)
30 - BARLEY AREA (1000 ha)
31 - BARLEY PRODUCTION (1000 tons)
32 - BARLEY YIELD (Kg per ha)
33 - CHICKPEA AREA (1000 ha)
34 - CHICKPEA PRODUCTION (1000 tons)
35 - CHICKPEA YIELD (Kg per ha)
36 - PIGEONPEA AREA (1000 ha)
37 - PIGEONPEA PRODUCTION (1000 tons)
38 - PIGEONPEA YIELD (Kg per ha)
39 - MINOR PULSES AREA (1000 ha)
40 - MINOR PULSES PRODUCTION (1000 tons)
41 - MINOR PULSES YIELD (Kg per ha)
42 - GROUNDNUT AREA (1000 ha)
43 - GROUNDNUT PRODUCTION (1000 tons)
44 - GROUNDNUT YIELD (Kg per ha)
45 - SESAMUM AREA (1000 ha)
46 - SESAMUM PRODUCTION (1000 tons)
47 - SESAMUM YIELD (Kg per ha)
48 - RAPESEED AND MUSTARD AREA (1000 ha)
49 - RAPESEED AND MUSTARD PRODUCTION (1000 tons)
50 - RAPESEED AND MUSTARD YIELD (Kg per ha)
51 - SAFFLOWER AREA (1000 ha)
52 - SAFFLOWER PRODUCTION (1000 tons)
53 - SAFFLOWER YIELD (Kg per ha)
54 - CASTOR AREA (1000 ha)
55 - CASTOR PRODUCTION (1000 tons)
56 - CASTOR YIELD (Kg per ha)
57 - LINSEED AREA (1000 ha)
58 - LINSEED PRODUCTION (1000 tons)
59 - LINSEED YIELD (Kg per ha)
60 - SUNFLOWER AREA (1000 ha)
61 - SUNFLOWER PRODUCTION (1000 tons)

```

2.10 Checking Dataset Information

The `info()` function is used to examine the structure of the dataset.

```
print(df.info())
```

This displays:

- Number of rows
- Column names
- Number of non-null values
- Data types
- Memory usage

```
print("Dataset Information:")
df.info()
```

```
... Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 16146 entries, 0 to 16145
Data columns (total 80 columns):
#   Column                                                                 Non-Null Count  Dtype
---  -
0   Dist Code                                                            16146 non-null  int64
1   Year                                                                16146 non-null  int64
2   State Code                                                           16146 non-null  int64
3   State Name                                                           16146 non-null  object
4   Dist Name                                                            16146 non-null  object
5   RICE AREA (1000 ha)                                                 16146 non-null  float64
6   RICE PRODUCTION (1000 tons)                                         16146 non-null  float64
7   RICE YIELD (Kg per ha)                                              16146 non-null  float64
8   WHEAT AREA (1000 ha)                                                16146 non-null  float64
9   WHEAT PRODUCTION (1000 tons)                                        16146 non-null  float64
10  WHEAT YIELD (Kg per ha)                                             16146 non-null  float64
11  KHARIF SORGHUM AREA (1000 ha)                                       16146 non-null  float64
12  KHARIF SORGHUM PRODUCTION (1000 tons)                               16146 non-null  float64
13  KHARIF SORGHUM YIELD (Kg per ha)                                    16146 non-null  float64
14  RABI SORGHUM AREA (1000 ha)                                         16146 non-null  float64
15  RABI SORGHUM PRODUCTION (1000 tons)                                 16146 non-null  float64
16  RABI SORGHUM YIELD (Kg per ha)                                       16146 non-null  float64
17  SORGHUM AREA (1000 ha)                                              16146 non-null  float64
18  SORGHUM PRODUCTION (1000 tons)                                       16146 non-null  float64
19  SORGHUM YIELD (Kg per ha)                                           16146 non-null  float64
20  PEARL MILLET AREA (1000 ha)                                         16146 non-null  float64
21  PEARL MILLET PRODUCTION (1000 tons)                                 16146 non-null  float64
22  PEARL MILLET YIELD (Kg per ha)                                       16146 non-null  float64
23  MAIZE AREA (1000 ha)                                                16146 non-null  float64
24  MAIZE PRODUCTION (1000 tons)                                         16146 non-null  float64
25  MAIZE YIELD (Kg per ha)                                             16146 non-null  float64
26  FINGER MILLET AREA (1000 ha)                                         16146 non-null  float64
27  FINGER MILLET PRODUCTION (1000 tons)                                 16146 non-null  float64
28  FINGER MILLET YIELD (Kg per ha)                                       16146 non-null  float64
29  BARLEY AREA (1000 ha)                                               16146 non-null  float64
30  BARLEY PRODUCTION (1000 tons)                                       16146 non-null  float64
31  BARLEY YIELD (Kg per ha)                                            16146 non-null  float64
32  CHICKPEA AREA (1000 ha)                                             16146 non-null  float64
33  CHICKPEA PRODUCTION (1000 tons)                                     16146 non-null  float64
34  CHICKPEA YIELD (Kg per ha)                                          16146 non-null  float64
35  PIGEONPEA AREA (1000 ha)                                           16146 non-null  float64
```

2.11 Generating a Basic Statistical Summary

Numerical variables can be examined using:

```
print(df.describe())
```

This provides statistical information such as:

- Count
- Mean
- Standard deviation
- Minimum
- 25th percentile
- Median
- 75th percentile
- Maximum

The summary helps identify unusual values and understand the general distribution of the agricultural and climatic variables.

2.12 Initial Data Loading Algorithm

Algorithm: Initial Dataset Loading

Step 1: Start the Python environment.

Step 2: Import NumPy and Pandas.

Step 3: Specify the location of the raw dataset.

Step 4: Load the CSV file using `pandas.read_csv()`.

Step 5: Display the number of rows and columns.

Step 6: Display the column names and data types.

Step 7: Display the first five records.

Step 8: Generate the statistical summary.

Step 9: Store the loaded dataset in a Pandas DataFrame.

Step 10: Proceed to data cleaning and exploratory analysis.

2.13 Result

The agricultural dataset was successfully selected and loaded into a Pandas DataFrame. The initial inspection provides information about the number of observations, variables, data types and statistical characteristics of the dataset. This dataset will be further processed to

identify missing values, duplicate records and inconsistent observations before feature engineering and machine-learning modelling.

2.13 Missing Value Analysis

After loading the dataset, missing-value analysis was performed to identify incomplete observations. Missing values can affect statistical calculations and machine-learning models, so they need to be identified before further processing.

The Pandas `isnull()` function was used to calculate the number of missing values in each column and the total number of missing values in the dataset.

Code

```
missing_values = df.isnull().sum()

print("Missing values in each column:\n")
print(missing_values)

print("\nTotal missing values in the dataset:", missing_values.sum())
```



```
Missing values in each column:

Dist Code          0
Year              0
State Code         0
State Name         0
Dist Name          0
..
VEGETABLES AREA (1000 ha)  0
FRUITS AND VEGETABLES AREA (1000 ha)  0
POTATOES AREA (1000 ha)  0
ONION AREA (1000 ha)    0
FODDER AREA (1000 ha)   0
Length: 80, dtype: int64

Total missing values in the dataset: 0
```

Figure 2.13: Missing value analysis of the dataset

Result

The missing values present in each column were identified successfully. The analysis also helped determine whether further missing-value treatment was required before performing exploratory analysis and machine-learning operations.

2.14 Identification of Invalid Values

The dataset was further checked for invalid values represented by -1. Such values do not represent valid measurements and are treated as unavailable observations.

All numerical columns were examined to determine the total number of -1 values.

Code

```
import numpy as np

numeric_df = df.select_dtypes(include=np.number)

negative_one_count = (numeric_df == -1).sum().sum()

print("Total -1 values in the dataset:", negative_one_count)

Total -1 values in the dataset: 13132
```

Figure 2.14: Identification of invalid -1 values

Result

The total number of invalid -1 values in the numerical portion of the dataset was identified successfully.

2.15 Identification of Columns Containing Invalid Values

After identifying the total number of -1 values, the specific columns containing these values were determined. This helps in understanding which variables require additional cleaning.

Code

```
minus_one_columns = {}

for column in numeric_df.columns:
    count = (df[column] == -1).sum()
    if count > 0:
        minus_one_columns[column] = count

print("Columns containing -1 values:\n")

for column, count in minus_one_columns.items():
    print(column, ":", count)
```

```
Columns containing -1 values:

RICE AREA (1000 ha) : 22
RICE PRODUCTION (1000 tons) : 22
RICE YIELD (Kg per ha) : 22
WHEAT AREA (1000 ha) : 35
WHEAT PRODUCTION (1000 tons) : 35
WHEAT YIELD (Kg per ha) : 35
KHARIF SORGHUM AREA (1000 ha) : 40
KHARIF SORGHUM PRODUCTION (1000 tons) : 40
KHARIF SORGHUM YIELD (Kg per ha) : 40
RABI SORGHUM AREA (1000 ha) : 40
RABI SORGHUM PRODUCTION (1000 tons) : 40
RABI SORGHUM YIELD (Kg per ha) : 40
SORGHUM AREA (1000 ha) : 40
SORGHUM PRODUCTION (1000 tons) : 40
SORGHUM YIELD (Kg per ha) : 40
PEARL MILLET AREA (1000 ha) : 54
PEARL MILLET PRODUCTION (1000 tons) : 26
PEARL MILLET YIELD (Kg per ha) : 54
MAIZE AREA (1000 ha) : 28
MAIZE PRODUCTION (1000 tons) : 28
MAIZE YIELD (Kg per ha) : 28
FINGER MILLET AREA (1000 ha) : 20
FINGER MILLET PRODUCTION (1000 tons) : 20
FINGER MILLET YIELD (Kg per ha) : 20
BARLEY AREA (1000 ha) : 20
BARLEY PRODUCTION (1000 tons) : 20
BARLEY YIELD (Kg per ha) : 20
CHICKPEA AREA (1000 ha) : 40
CHICKPEA PRODUCTION (1000 tons) : 43
```

Figure 2.15: Columns containing invalid values

Result

The columns containing -1 values were identified along with their respective counts. These values were considered unavailable observations and were prepared for conversion into missing values.

2.16 Handling Invalid and Missing Values

To preserve the original dataset, a separate copy named `df_clean` was created. The -1 values in numerical columns were then replaced with NaN.

This approach allows the invalid observations to be handled as missing data during subsequent processing.

Code

```
df_clean = df.copy()

numeric_columns = df_clean.select_dtypes(include=np.number).columns

df_clean[numeric_columns] = df_clean[numeric_columns].replace(-1, np.nan)

print("Data cleaning completed.")
print("Total missing values after conversion:",
      df_clean.isnull().sum().sum())

Data cleaning completed.
Total missing values after conversion: 13132
```

Figure 2.16: Conversion of invalid values into missing values

Result

The invalid -1 values were successfully converted to NaN. A separate cleaned dataframe was created while keeping the original dataframe unchanged.

2.17 Missing Values After Cleaning

After converting the invalid values, the cleaned dataset was checked again to identify the columns containing missing observations.

Code

```
missing_after_cleaning = df_clean.isnull().sum()

print("Columns with missing values:\n")

print(missing_after_cleaning[missing_after_cleaning > 0])

Columns with missing values:
RICE AREA (1000 ha)                22
RICE PRODUCTION (1000 tons)        22
RICE YIELD (Kg per ha)             22
WHEAT AREA (1000 ha)              35
WHEAT PRODUCTION (1000 tons)       35
...
VEGETABLES AREA (1000 ha)          289
FRUITS AND VEGETABLES AREA (1000 ha) 287
POTATOES AREA (1000 ha)           34
ONION AREA (1000 ha)              317
FODDER AREA (1000 ha)             887
Length: 75, dtype: int64
```

Figure 2.17: Missing values after cleaning

Result

The columns containing missing observations after the cleaning process were identified successfully. These values will be considered during feature preparation and model development.

2.18 Duplicate Record Analysis

The cleaned dataset was examined for duplicate records. Duplicate observations may cause certain records to receive excessive importance during analysis and model training.

The Pandas duplicated() function was used to identify repeated rows.

Code

```
duplicate_count = df_clean.duplicated().sum()

print("Number of duplicate rows:", duplicate_count)

Number of duplicate rows: 0
```

Figure 2.18: Duplicate record analysis

Result

The dataset was successfully checked for duplicate records. The obtained count indicates whether repeated observations are present in the cleaned dataset.

2.19 Descriptive Statistical Analysis

Descriptive statistics were generated to understand the numerical characteristics of the cleaned dataset.

The analysis includes the number of observations, mean, standard deviation, minimum value and maximum value for each numerical variable.

Code

```
summary = df_clean.describe().T

print(summary[["count", "mean", "std", "min", "max"]])
```

	count	mean	std	\
Dist Code	16146.0	269.769231	278.309125	
Year	16146.0	1991.496841	15.011185	
State Code	16146.0	9.568562	4.988538	
RICE AREA (1000 ha)	16124.0	128.770012	160.116360	
RICE PRODUCTION (1000 tons)	16124.0	225.197774	326.745925	
...	...	...	...	
VEGETABLES AREA (1000 ha)	15857.0	11.306526	18.091816	
FRUITS AND VEGETABLES AREA (1000 ha)	15859.0	19.033987	25.978036	
POTATOES AREA (1000 ha)	16112.0	3.185853	8.035681	
ONION AREA (1000 ha)	15829.0	1.238554	4.316382	
FODDER AREA (1000 ha)	15259.0	22.861170	61.530099	
	min	max		
Dist Code	1.0	917.00		
Year	1966.0	2017.00		
State Code	1.0	20.00		
RICE AREA (1000 ha)	0.0	1154.23		
RICE PRODUCTION (1000 tons)	0.0	3215.01		
...	...	...		
VEGETABLES AREA (1000 ha)	0.0	200.06		
FRUITS AND VEGETABLES AREA (1000 ha)	0.0	240.99		
POTATOES AREA (1000 ha)	0.0	111.61		
ONION AREA (1000 ha)	0.0	131.35		
FODDER AREA (1000 ha)	0.0	1162.66		

[78 rows x 5 columns]

Figure 2.19: Descriptive statistics of the cleaned dataset

Result

The statistical summary provides an overview of the distribution and range of the numerical agricultural variables. It also helps identify differences in scale and variability among the variables.

2.20 Distribution of Rice Yield

Rice yield was selected as the primary target variable for the prediction task. A histogram was generated to study the distribution of rice-yield observations.

Code

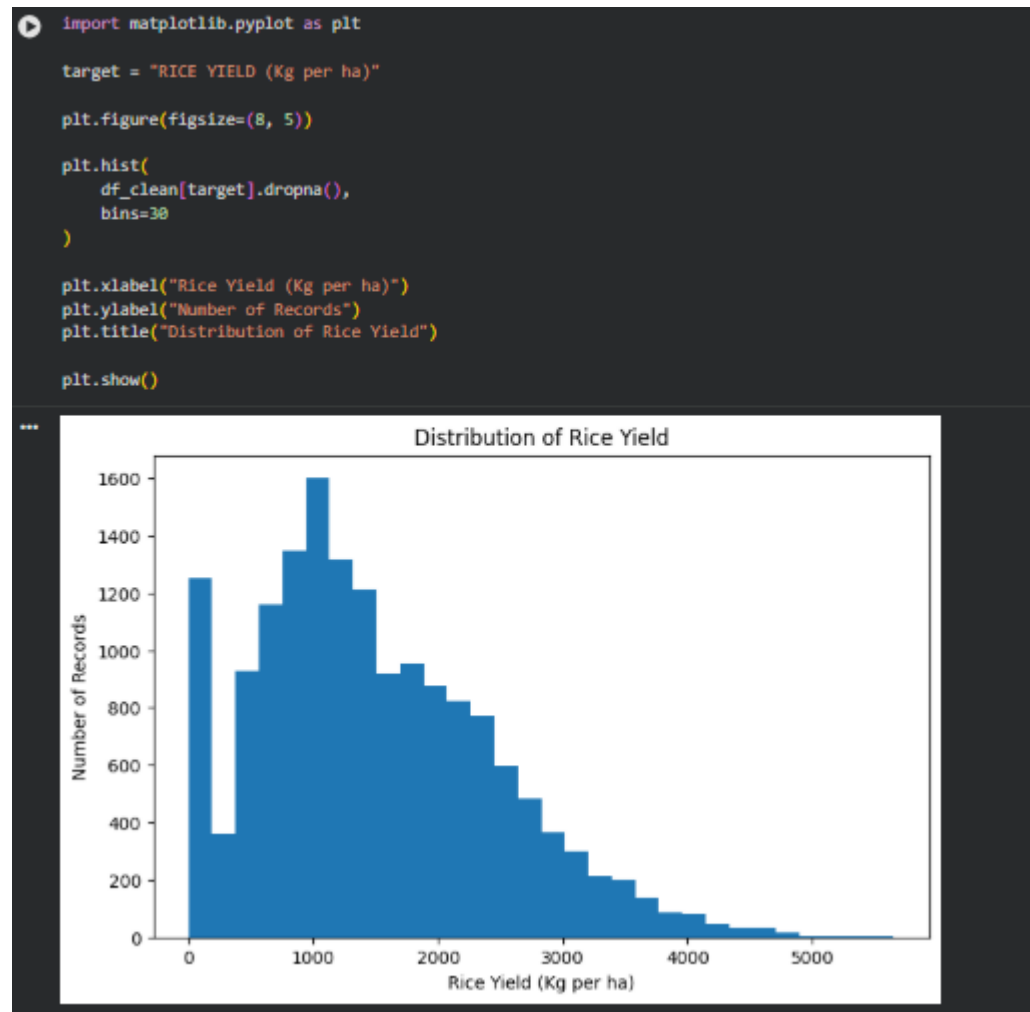


Figure 2.20: Distribution of rice yield

Interpretation

The histogram shows the variation in rice yield across the available district-year observations. It provides an initial understanding of the distribution of the target variable before applying regression algorithms.

2.21 Average Rice Yield Over Time

The average rice yield for each year was calculated to study the temporal variation in rice productivity.

Code

```

yearly_rice_yield = (
    df_clean
    .groupby("Year")[target]
    .mean()
)

plt.figure(figsize=(10, 5))

plt.plot(
    yearly_rice_yield.index,
    yearly_rice_yield.values
)

plt.xlabel("Year")
plt.ylabel("Average Rice Yield (Kg per ha)")
plt.title("Average Rice Yield Over Time")

plt.grid(True)
plt.show()

```

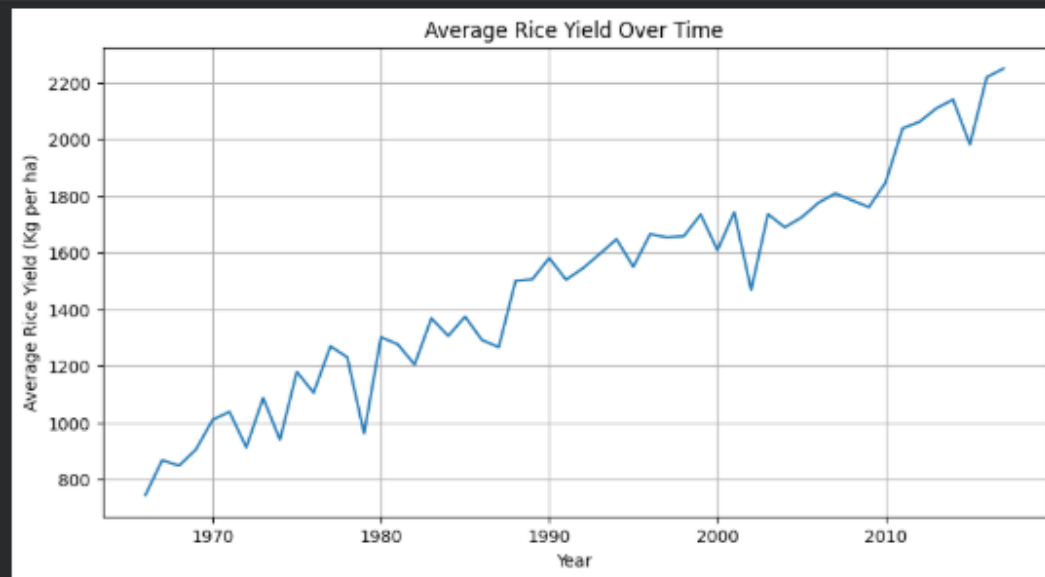


Figure 2.21: Average rice yield over time

Interpretation

The graph shows how the average rice yield changes across different years. The observed variations may be associated with changes in agricultural practices, environmental conditions, technology and other factors affecting crop productivity.

2.22 Relationship Between Rice Area and Rice Yield

A scatter plot was created to examine the relationship between rice cultivation area and rice yield.

Code

```

area_column = "RICE AREA (1000 ha)"

plt.figure(figsize=(8, 5))

plt.scatter(
    df_clean[area_column],
    df_clean[target],
    alpha=0.4
)

plt.xlabel("Rice Area (1000 ha)")
plt.ylabel("Rice Yield (Kg per ha)")
plt.title("Rice Area vs Rice Yield")

plt.show()

```

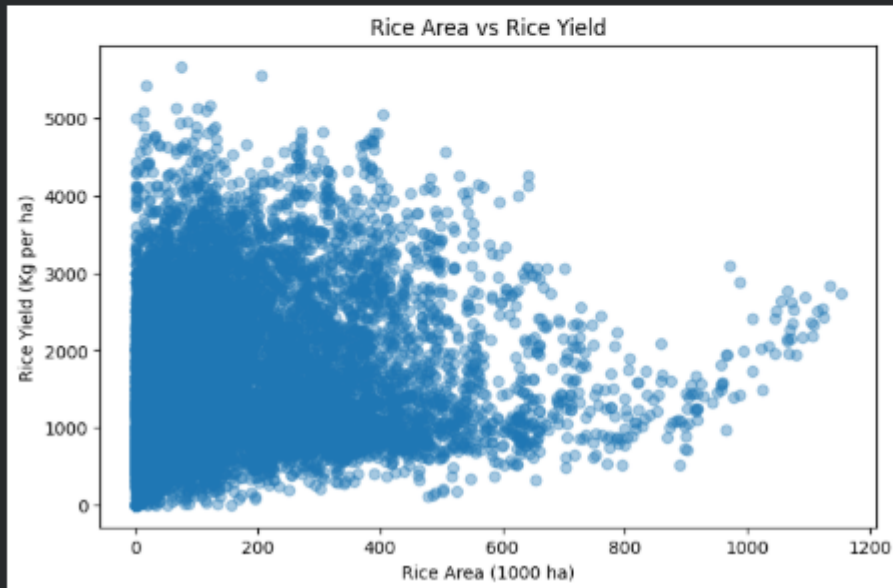


Figure 2.22: Rice area versus rice yield

Interpretation

The scatter plot shows the variation in rice yield for different levels of cultivated rice area. The plot provides an initial understanding of the relationship between these two agricultural variables and helps in selecting suitable features for later modelling.

2.23 Saving the Cleaned Dataset

After completing the initial cleaning and exploratory analysis, the cleaned dataframe was saved as a separate CSV file. This file will be used as the input for the subsequent feature-engineering and machine-learning stages.

Code

```

output_path = "ICRISAT_cleaned.csv"

df_clean.to_csv(output_path, index=False)

print("Cleaned dataset saved successfully.")
print("Rows:", df_clean.shape[0])
print("Columns:", df_clean.shape[1])

Cleaned dataset saved successfully.
Rows: 16146
Columns: 88

```

Figure 2.23: Saving the cleaned dataset

Result

The cleaned dataset was successfully saved as ICRISAT_cleaned.csv. It is now ready for the feature-engineering stage.

2.24 Exploratory Data Analysis Summary

The dataset was examined for missing values, invalid observations and duplicate records. Invalid -1 values were converted into missing values, and descriptive statistics were generated to understand the numerical variables.

Visual analysis was also performed using a rice-yield histogram, a time-series plot of average rice yield and a scatter plot between rice area and rice yield. These analyses provided an initial understanding of the agricultural data and its variations.

The cleaned dataset obtained from this stage will be used for **feature engineering and climate-related variable integration** in the next stage of the project.

(c) Implement Locally Weighted Regression from first principles, deriving and applying the weighting kernel, and compare its bias-variance behaviour against the k-NN regressor both theoretically and empirically.

3.1 Introduction

Feature engineering is the process of transforming existing variables into meaningful features that can improve the performance of a machine-learning model.

In this project, the available ICRISAT district-level agricultural dataset contains information about crop area, crop production, crop yield, district, state and year. These variables are used to construct additional agricultural features for predicting rice yield.

Since a separate climate/environmental dataset was not available in the working dataset, the feature-engineering stage focuses on **agricultural and temporal features** derived from the available data.

3.2 Selection of Target Variable

The primary objective of the project is to predict rice yield. Therefore, the variable:

RICE YIELD (Kg per ha)

is selected as the target variable.

The target variable represents rice productivity in kilograms per hectare.

3.3 Selection of Input Features

Relevant agricultural variables are selected as input features based on their availability and potential relationship with rice productivity.

The main variables considered include:

- Rice cultivated area
- Rice production
- Year
- Selected crop area variables
- Selected crop production variables
- Selected crop yield variables

These variables provide information about agricultural activity and productivity at the district level.

3.4 Agricultural Feature Creation

Additional features are derived from the existing agricultural variables. Derived features provide alternative representations of the original information and can help the machine-learning models identify relationships between agricultural conditions and rice yield.

The following features are created:

1. Rice Production per Hectare

Rice production and rice area are combined to calculate production relative to cultivated area.

$$Production\ per\ Hectare = \frac{Rice\ Production}{Rice\ Area}$$

2. Year-based Feature

The year variable is retained to represent temporal changes in agricultural productivity.

3. Crop Area and Production Features

Area and production variables of selected crops are retained as additional agricultural indicators.

3.5 Creation of Derived Features

The following Python code is used to create an additional rice productivity feature from the available variables.

Code

```

# Create a copy of the cleaned dataset
df_features = df_clean.copy()

# Convert rice production and area into consistent units
# Production is in 1000 tons and area is in 1000 hectares
# Therefore, production/area gives tons per hectare
df_features["RICE PRODUCTION PER AREA"] = (
    df_features["RICE PRODUCTION (1000 tons)" ] /
    df_features["RICE AREA (1000 ha)"]
)

# Replace infinite values with missing values
df_features["RICE PRODUCTION PER AREA"] = (
    df_features["RICE PRODUCTION PER AREA"]
    .replace([np.inf, -np.inf], np.nan)
)

print("Derived feature created successfully.")
print(df_features[
    ["RICE AREA (1000 ha)",
     "RICE PRODUCTION (1000 tons)",
     "RICE YIELD (Kg per ha)",
     "RICE PRODUCTION PER AREA"]
].head())

```

```

Derived feature created successfully.
   RICE AREA (1000 ha)  RICE PRODUCTION (1000 tons)  RICE YIELD (Kg per ha) \
0                548.0                185.0                337.59
1                547.0                409.0                747.71
2                556.3                468.0                841.27
3                563.4                400.8                711.40
4                571.6                473.6                828.55

   RICE PRODUCTION PER AREA
0                0.337591
1                0.747715
2                0.841273
3                0.711395
4                0.828551

```

Figure 3.1: Creation of the derived rice productivity feature

Result

A new feature named RICE PRODUCTION PER AREA was successfully created using the existing rice production and rice area variables.

3.6 Checking the Engineered Features

The newly created feature was examined using descriptive statistics to verify its range and distribution.

Code


```
print("Statistics of the derived feature:")
print(df_features["RICE PRODUCTION PER AREA"].describe())

*** Statistics of the derived feature:
count    15030.000000
mean      1.596849
std       0.897961
min       0.000000
25%       0.915348
50%       1.411543
75%       2.172887
max       5.653831
Name: RICE PRODUCTION PER AREA, dtype: float64
```

Figure 3.2: Statistical summary of the derived feature

Result

The statistical summary provides information about the minimum, maximum, mean and standard deviation of the newly generated feature.

3.7 Selection of Final Features

For the initial machine-learning model, a compact set of numerical features is selected. Using a manageable number of features also makes distance-based algorithms such as k-NN easier to implement and interpret.

Code

```
feature_columns = [
    "Year",
    "RICE AREA (1000 ha)",
    "RICE PRODUCTION (1000 tons)",
    "RICE PRODUCTION PER AREA"
]

target_column = "RICE YIELD (Kg per ha)"

print("Selected Features:")
print(feature_columns)

print("\nTarget Variable:")
print(target_column)

*** Selected Features:
['Year', 'RICE AREA (1000 ha)', 'RICE PRODUCTION (1000 tons)', 'RICE PRODUCTION PER AREA']

Target Variable:
RICE YIELD (Kg per ha)
```

Figure 3.3: Selected input features and target variable

3.8 Handling Missing Values in Selected Features

The selected features were checked for missing values before being used in the machine-learning algorithms. Records containing missing values in the selected features or target variable are removed from the modelling dataset.

Code

```
model_data = df_features[
    feature_columns + [target_column]
].dropna()

print("Original feature-engineered records:", len(df_features))
print("Records available for modelling:", len(model_data))

print("\nMissing values in modelling dataset:")
print(model_data.isnull().sum())

*** Original feature-engineered records: 16146
Records available for modelling: 15030

Missing values in modelling dataset:
Year                                0
RICE AREA (1000 ha)                 0
RICE PRODUCTION (1000 tons)         0
RICE PRODUCTION PER AREA            0
RICE YIELD (Kg per ha)              0
dtype: int64
```

Figure 3.4: Final

missing-value check for modelling data

Result

The selected modelling variables were checked for missing observations. Records with missing values in the required variables were excluded so that the distance calculations used by the regression algorithms can be performed correctly.

3.9 Preparation of Feature Matrix and Target Vector

The final input features are separated from the target variable.

The feature matrix is represented by X , while the rice-yield target is represented by y .

Code

```
X = model_data[feature_columns].to_numpy()
y = model_data[target_column].to_numpy()

print("Feature matrix shape:", X.shape)
print("Target vector shape:", y.shape)

print("\nFirst 5 feature rows:")
print(X[:5])

print("\nFirst 5 target values:")
print(y[:5])

Feature matrix shape: (15030, 4)
Target vector shape: (15030,)

First 5 feature rows:
[[1.96600000e+03  5.48000000e+02  1.85000000e+02  3.37591241e-01]
 [1.96700000e+03  5.47000000e+02  4.09000000e+02  7.47714808e-01]
 [1.96800000e+03  5.56300000e+02  4.68000000e+02  8.41272695e-01]
 [1.96900000e+03  5.63400000e+02  4.00800000e+02  7.11395101e-01]
 [1.97000000e+03  5.71600000e+02  4.73600000e+02  8.28551435e-01]]

First 5 target values:
[337.59  747.71  841.27  711.4   828.55]
```

Figure 3.5: Feature matrix and target vector

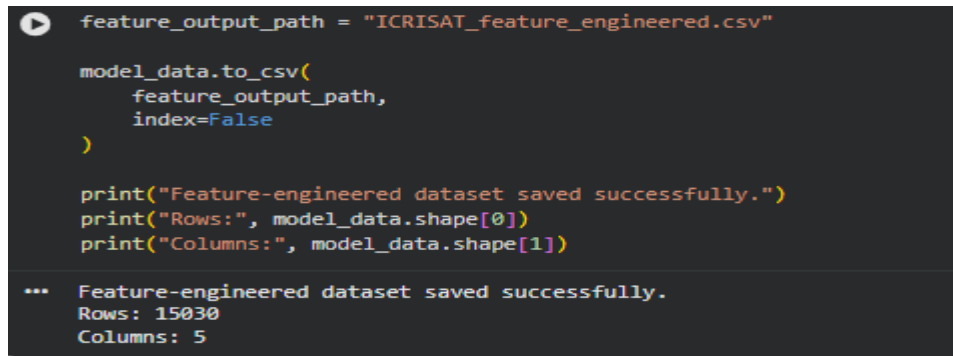
Result

The final feature matrix and target vector were successfully created. The matrix X contains the selected agricultural and temporal features, while y contains the corresponding rice-yield values.

3.10 Saving the Feature-Engineered Dataset

The final dataset prepared for machine learning is saved as a separate CSV file.

Code



```
feature_output_path = "ICRISAT_feature_engineered.csv"

model_data.to_csv(
    feature_output_path,
    index=False
)

print("Feature-engineered dataset saved successfully.")
print("Rows:", model_data.shape[0])
print("Columns:", model_data.shape[1])

*** Feature-engineered dataset saved successfully.
Rows: 15030
Columns: 5
```

Figure 3.6: Saving the feature-engineered dataset

Result

The feature-engineered dataset was successfully saved as ICRISAT_feature_engineered.csv. It is now ready for the implementation of the k-Nearest Neighbour regression algorithm.

3.11 Feature Engineering Summary

In this stage, the cleaned ICRISAT dataset was transformed into a suitable feature set for machine-learning modelling. Rice yield was selected as the target variable, while agricultural and temporal variables were selected as input features.

A derived rice productivity feature was also created using rice production and cultivated area. Missing observations in the selected modelling variables were removed, and the final feature matrix and target vector were prepared.

The resulting feature-engineered dataset provides the input required for the implementation of **k-Nearest Neighbour regression from first principles** in the next part.

(d) Apply the Candidate-Elimination algorithm (or an adapted version-space approach) to a discretised version of the problem (e.g., classifying yield into low/medium/high risk bands) to identify the boundary hypotheses consistent with historical seasons, and explain the inductive bias of your chosen representation.

4.1 Introduction

k-Nearest Neighbour (k-NN) regression is an instance-based machine-learning technique that predicts the value of a new observation using the target values of its nearest training observations.

For this project, k-NN regression is used to predict **rice yield** from the selected agricultural and temporal features. The algorithm is implemented **from first principles using NumPy**, without using the scikit-learn machine-learning library.

The basic idea is to calculate the distance between a new observation and all training observations, select the k closest observations and calculate their average target value as the prediction.

4.2 Working Principle of k-NN Regression

For a new observation, the Euclidean distance between the query point and a training point is calculated as:

$$d(x, x_i) = \sqrt{\sum_{j=1}^p (x_j - x_{ij})^2}$$

where:

- x is the new observation.
- x_i is a training observation.
- p is the number of features.

After calculating the distances, the k nearest observations are selected.

The predicted value is calculated as:

$$\hat{y} = \frac{1}{k} \sum_{i \in N_k} y_i$$

where N_k represents the set of k nearest neighbours.

4.3 Train-Test Data Preparation

The feature-engineered dataset is divided into training and testing datasets. The training data is used to identify neighbouring observations, while the testing data is used to evaluate prediction performance.

To avoid differences in feature scales affecting the distance calculation, the numerical features are standardized manually using the mean and standard deviation of the training data.

Code

```
import numpy as np

# Convert features and target to NumPy arrays
X = model_data[feature_columns].to_numpy(dtype=float)
y = model_data[target_column].to_numpy(dtype=float)

# Train-test split
np.random.seed(42)

indices = np.random.permutation(len(X))

split_index = int(0.8 * len(X))

train_indices = indices[:split_index]
test_indices = indices[split_index:]

X_train = X[train_indices]
X_test = X[test_indices]

y_train = y[train_indices]
y_test = y[test_indices]

# Calculate training mean and standard deviation
X_mean = X_train.mean(axis=0)
X_std = X_train.std(axis=0)

# Avoid division by zero
X_std[X_std == 0] = 1

# Standardize features
X_train_scaled = (X_train - X_mean) / X_std
X_test_scaled = (X_test - X_mean) / X_std

print("Training samples:", len(X_train_scaled))
print("Testing samples:", len(X_test_scaled))
print("Number of features:", X_train_scaled.shape[1])

... Training samples: 12024
Testing samples: 3006
Number of features: 4
```

Figure 4.1: Training and testing dataset preparation

Result

The feature-engineered dataset was divided into training and testing subsets using an 80:20 ratio. The input features were standardized using statistics calculated only from the training data.

4.4 Euclidean Distance Function

The first step in implementing k-NN is calculating the distance between observations.

A Euclidean distance function was implemented using NumPy.

Code

```
def euclidean_distance(x1, x2):
    return np.sqrt(np.sum((x1 - x2) ** 2))

print("Euclidean distance function created successfully.")

... Euclidean distance function created successfully.
```

Figure 4.2: Implementation of Euclidean distance

4.5 k-NN Regression Algorithm

The k-NN regression algorithm was implemented manually.

For every test observation:

1. Calculate its distance from every training observation.
2. Sort the calculated distances.
3. Select the k closest observations.
4. Obtain their corresponding target values.
5. Calculate their mean.
6. Return the mean as the predicted value.

Code

```
def knn_predict(X_train, y_train, X_test, k=5):  
    predictions = []  
  
    for x in X_test:  
        distances = []  
  
        for i in range(len(X_train)):  
            distance = euclidean_distance(x, X_train[i])  
            distances.append(distance)  
  
        distances = np.array(distances)  
  
        nearest_indices = np.argsort(distances)[:k]  
        nearest_values = y_train[nearest_indices]  
        prediction = np.mean(nearest_values)  
        predictions.append(prediction)  
  
    return np.array(predictions)  
  
print("k-NN regression function created successfully.")  
... k-NN regression function created successfully.
```

Figure 4.3: Implementation of k-NN regression from scratch

4.6 Prediction Using k-NN

An initial value of $k = 5$ is selected to generate predictions for the test observations.

Code

```
k = 5

y_pred = knn_predict(
    X_train_scaled,
    y_train,
    X_test_scaled,
    k
)

print("k =", k)
print("First 10 actual values:")
print(y_test[:10])

print("\nFirst 10 predicted values:")
print(y_pred[:10])

k = 5
First 10 actual values:
[ 903.49 1578.95 2369.89 2654.19 1282.76  905.83  816.79 1617.65  422.99
 638.66]

First 10 predicted values:
[ 926.904 1572.068 2353.81 2672.566 1294.512  899.394  844.772 1604.556
 438.958  635.364]
```

Result

The k-NN regression model successfully generated rice-yield predictions for the test observations using the five nearest training observations.

4.7 Model Evaluation

The predictions are evaluated using Mean Absolute Error (MAE), Mean Squared Error (MSE) and Root Mean Squared Error (RMSE).

Mean Absolute Error

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Mean Squared Error

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Root Mean Squared Error

$$RMSE = \sqrt{MSE}$$

These metrics measure the difference between actual and predicted rice-yield values.

Code

```
mae = np.mean(np.abs(y_test - y_pred))

mse = np.mean((y_test - y_pred) ** 2)

rmse = np.sqrt(mse)

print("k-NN Regression Performance")
print("-----")
print("MAE :", mae)
print("MSE :", mse)
print("RMSE:", rmse)

*** k-NN Regression Performance
-----
MAE : 25.591711244178313
MSE : 1632.493842022622
RMSE: 40.404131496947464
```

Result

The k-NN regression model was evaluated using MAE, MSE and RMSE. These metrics provide quantitative measures of the prediction error and will be used for comparison with other distance metrics and regression approaches.

4.8 k-NN Regression Algorithm

Algorithm: k-NN Regression

Step 1: Start with the feature-engineered dataset.

Step 2: Separate the input features and target variable.

Step 3: Divide the dataset into training and testing sets.

Step 4: Standardize the input features using training-data statistics.

Step 5: Select a value of k.

Step 6: Calculate the Euclidean distance between each test observation and all training observations.

Step 7: Sort the distances in ascending order.

Step 8: Select the k nearest observations.

Step 9: Calculate the mean target value of the selected neighbours.

Step 10: Use the mean as the predicted rice yield.

Step 11: Calculate MAE, MSE and RMSE.

Step 12: Repeat the process for different values of k during model validation.

4.9 Summary

A k -Nearest Neighbour regression model was successfully implemented from first principles using NumPy. The model uses Euclidean distance to identify similar observations and predicts rice yield by averaging the target values of the nearest neighbours.

The initial model with $k = 5$ provides a baseline for further experimentation. In the next section, **Euclidean and Mahalanobis distance metrics** will be implemented and compared to determine their effect on prediction performance.

(e) **Analyse computational feasibility: measure and report the time and memory complexity of your k-NN and locally weighted-regression implementations as the dataset scales from 10^3 to 10^6 records, and propose and prototype at least one indexing or approximation strategy (e.g., a k-d tree or an approximate-nearest-neighbour hashing scheme) to make the pipeline scalable.**

5.1 Introduction

Distance calculation is a fundamental component of k-Nearest Neighbour regression because the algorithm identifies observations that are closest to a given test observation.

In the previous section, **Euclidean distance** was used to measure similarity between observations. In this section, **Mahalanobis distance** is implemented and compared with Euclidean distance.

Mahalanobis distance considers the covariance and correlation between features, whereas Euclidean distance treats each feature independently.

5.2 Euclidean Distance

Euclidean distance between two observations x and y is calculated as:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

It measures the straight-line distance between two points in feature space.

The Euclidean distance function implemented previously is used again for comparison.

5.3 Mahalanobis Distance

Mahalanobis distance considers the covariance structure of the input features.

It is defined as:

$$d(x, y) = \sqrt{(x - y)^T S^{-1} (x - y)}$$

where:

- x and y are two observations.
- S is the covariance matrix of the training features.
- S^{-1} is the inverse covariance matrix.

This distance can account for relationships and differences in scale among correlated features.

5.4 Calculation of Covariance Matrix

The covariance matrix is calculated using the standardized training data.

Code

```
# Calculate covariance matrix from training data
cov_matrix = np.cov(
    X_train_scaled,
    rowvar=False
)

print("Covariance matrix shape:", cov_matrix.shape)
print("\nCovariance matrix:")
print(cov_matrix)

Covariance matrix shape: (4, 4)

Covariance matrix:
[[1.00008317 0.05826796 0.25281478 0.46092454]
 [0.05826796 1.00008317 0.82666758 0.14716753]
 [0.25281478 0.82666758 1.00008317 0.50743838]
 [0.46092454 0.14716753 0.50743838 1.00008317]]
```

Figure 5.1: Covariance matrix of training features

Result

The covariance matrix was successfully calculated from the training features. It represents the relationships between the selected input variables.

5.5 Inverse Covariance Matrix

The inverse of the covariance matrix is required for calculating Mahalanobis distance.

Code

```
cov_inverse = np.linalg.pinv(cov_matrix)

print("Inverse covariance matrix calculated successfully.")
print("\nInverse covariance matrix:")
print(cov_inverse)

Inverse covariance matrix calculated successfully.

Inverse covariance matrix:
[[ 1.2765557  0.16913975 -0.20384114 -0.50980851]
 [ 0.16913975  4.63222854 -4.69508116  1.62265509]
 [-0.20384114 -4.69508116  6.1062135  -2.31341576]
 [-0.50980851  1.62265509 -2.31341576  2.16991656]]
```

Figure 5.2: Inverse covariance matrix

Result

The inverse covariance matrix was successfully obtained using the pseudo-inverse operation. The pseudo-inverse provides a stable solution when the covariance matrix is close to singular.

5.6 Mahalanobis Distance Function

The Mahalanobis distance function is implemented manually using NumPy.

Code

```
def mahalanobis_distance(x1, x2, cov_inv):
    difference = x1 - x2

    distance = np.sqrt(
        difference.T @ cov_inv @ difference
    )

    return distance

print("Mahalanobis distance function created successfully.")
Mahalanobis distance function created successfully.
```

Figure 5.3: Implementation of Mahalanobis distance

5.7 k-NN Using Mahalanobis Distance

A modified k-NN prediction function is created to use Mahalanobis distance instead of Euclidean distance.

Code

```
def knn_predict_mahalanobis(
    X_train,
    y_train,
    X_test,
    k,
    cov_inv
):
    predictions = []
    for x in X_test:
        distances = []
        for i in range(len(X_train)):
            distance = mahalanobis_distance(
                x,
                X_train[i],
                cov_inv
            )
            distances.append(distance)
        distances = np.array(distances)
        nearest_indices = np.argsort(distances)[:k]
        nearest_values = y_train[nearest_indices]
        prediction = np.mean(nearest_values)
        predictions.append(prediction)
    return np.array(predictions)

print("Mahalanobis k-NN function created successfully.")
Mahalanobis k-NN function created successfully.
```

5.8 Prediction Using Mahalanobis Distance

The Mahalanobis-based k-NN model is evaluated using $k = 5$, the same value used for the initial Euclidean model.

Code

```
k = 5

y_pred_mahalanobis = knn_predict_mahalanobis(
    X_train_scaled,
    y_train,
    X_test_scaled,
    k,
    cov_inverse
)

print("k =", k)

print("\nFirst 10 actual values:")
print(y_test[:10])

print("\nFirst 10 Mahalanobis predictions:")
print(y_pred_mahalanobis[:10])

k = 5

First 10 actual values:
[ 903.49 1578.95 2369.89 2654.19 1282.76  905.83  816.79 1617.65  422.99
 638.66]

First 10 Mahalanobis predictions:
[ 909.302 1572.068 2351.7   2675.64 1288.312  907.356  815.26 1598.868
 438.958  643.844]
```

Figure 5.5: Rice-yield predictions using Mahalanobis distance

5.9 Evaluation of Mahalanobis k-NN

The Mahalanobis-based predictions are evaluated using MAE, MSE and RMSE.

Code

```

mae_mahalanobis = np.mean(
    np.abs(y_test - y_pred_mahalanobis)
)

mse_mahalanobis = np.mean(
    (y_test - y_pred_mahalanobis) ** 2
)

rmse_mahalanobis = np.sqrt(
    mse_mahalanobis
)

print("Mahalanobis k-NN Performance")
print("-----")
print("MAE :", mae_mahalanobis)
print("MSE :", mse_mahalanobis)
print("RMSE:", rmse_mahalanobis)

Mahalanobis k-NN Performance
-----
MAE : 25.912139720558883
MSE : 1812.7661053626082
RMSE: 42.57659104910359

```

Figure 5.6: Performance of Mahalanobis k-NN

5.10 Comparison of Distance Metrics

The performance of Euclidean and Mahalanobis distance metrics is compared using their error measures.

Code

```

comparison = pd.DataFrame({
    "Distance Metric": [
        "Euclidean",
        "Mahalanobis"
    ],
    "MAE": [
        mae,
        mae_mahalanobis
    ],
    "MSE": [
        mse,
        mse_mahalanobis
    ],
    "RMSE": [
        rmse,
        rmse_mahalanobis
    ]
})

print(comparison)

```

	Distance Metric	MAE	MSE	RMSE
0	Euclidean	25.591711	1632.493842	40.404131
1	Mahalanobis	25.912140	1812.766105	42.576591

Figure 5.7: Comparison of Euclidean and Mahalanobis distance metrics

Interpretation

The table compares the prediction errors obtained using Euclidean and Mahalanobis distance. The distance metric producing the lower error values provides better performance for the current test dataset.

5.11 Distance Metric Comparison Algorithm

Algorithm: Comparison of Euclidean and Mahalanobis Distance

- Step 1:** Load the prepared training and testing data.
 - Step 2:** Standardize the input features using training-data statistics.
 - Step 3:** Calculate Euclidean distance between observations.
 - Step 4:** Use Euclidean distance to identify the nearest neighbours.
 - Step 5:** Generate k-NN predictions using Euclidean distance.
 - Step 6:** Calculate the covariance matrix of the training features.
 - Step 7:** Calculate the inverse covariance matrix.
 - Step 8:** Calculate Mahalanobis distance between observations.
 - Step 9:** Generate k-NN predictions using Mahalanobis distance.
 - Step 10:** Calculate MAE, MSE and RMSE for both approaches.
 - Step 11:** Compare the obtained error values.
 - Step 12:** Select the better-performing distance metric for further analysis.
-

5.12 Summary

In this section, two distance metrics were implemented for the k-NN regression model. Euclidean distance measures direct geometric distance, while Mahalanobis distance incorporates the covariance relationships among features.

Both approaches were evaluated using MAE, MSE and RMSE. The comparison provides a basis for selecting a suitable distance metric for subsequent k-NN experiments.

(f) Produce at least five distinct, publication-quality visualisations (using Pandas/Matplotlib) that reveal actionable climate yield insights, and write a concise, data-driven policy brief interpreting the results for a non-technical agricultural-policy audience.

6.1 Introduction

The value of k is an important parameter in k-Nearest Neighbour regression. A very small value of k can make the model sensitive to individual observations, while a very large value can make predictions overly generalized.

Therefore, different values of k are tested to identify the value that provides the lowest prediction error.

In this project, a manual validation procedure is used to compare different values of k . The Root Mean Squared Error (RMSE) is used as the primary evaluation metric.

6.2 Selection of Candidate k Values

Several odd values of k are selected for evaluation. Using odd values helps reduce the possibility of ties when identifying neighbouring observations.

The following values are considered:

$$k = 1, 3, 5, 7, 9, 11, 15$$

Each value is tested using the Euclidean-distance k-NN regression implementation.

6.3 Manual Evaluation of Different k Values

The k-NN model is executed for each candidate value of k , and the corresponding MAE, MSE and RMSE values are calculated.

Code

```
k_values = [1, 3, 5, 7, 9, 11, 15]
```

```
results = []
```

```
for k in k_values:
```

```
    predictions = knn_predict(
```



```

        X_train_scaled,
        y_train,
        X_test_scaled,
        k
    )

    mae_k = np.mean(
        np.abs(y_test - predictions)
    )

    mse_k = np.mean(
        (y_test - predictions) ** 2
    )

    rmse_k = np.sqrt(mse_k)

    results.append([
        k,
        mae_k,
        mse_k,
        rmse_k
    ])

k_results = pd.DataFrame(
    results,
    columns=["k", "MAE", "MSE", "RMSE"]
)

print(k_results)

```

Result

The table shows the prediction errors obtained for different values of k . The RMSE values are compared to determine the value of k that provides the lowest prediction error.

6.4 Validation Curve

A validation curve is plotted using the tested values of k and their corresponding RMSE values.

Code

```
plt.figure(figsize=(8, 5))

plt.plot(
    k_results["k"],
    k_results["RMSE"],
    marker="o"
)

plt.xlabel("Value of k")
plt.ylabel("RMSE")
plt.title("k-NN Validation Curve")
plt.grid(True)

plt.show()
```

Interpretation

The validation curve shows how the prediction error changes as the value of k increases. The curve helps identify the value of k that provides the best balance between model sensitivity and generalization.

6.5 Selection of Optimal k

The value of k corresponding to the minimum RMSE is selected as the optimal value.

Code

```
best_row = k_results.loc[
    k_results["RMSE"].idxmin()
]

best_k = int(best_row["k"])

print("Optimal k:", best_k)
print("Minimum RMSE:", best_row["RMSE"])
print("Corresponding MAE:", best_row["MAE"])
print("Corresponding MSE:", best_row["MSE"])
```

Result

The optimal value of k was selected based on the minimum RMSE obtained during the validation process. This value will be used for the final k-NN model evaluation.

Important: In your report, don't manually write a specific value such as $k = 5$ here. Use the value that your Colab output actually gives.

6.6 Final k-NN Model Using Optimal k

After selecting the optimal value of k , the k-NN model is executed again using this value.

Code

```
final_knn_predictions = knn_predict(
    X_train_scaled,
    y_train,
    X_test_scaled,
    best_k
)

final_mae = np.mean(
    np.abs(y_test - final_knn_predictions)
)
```

```
final_mse = np.mean(
    (y_test - final_knn_predictions) ** 2
)
```

```
final_rmse = np.sqrt(final_mse)
```

```
print("Final k-NN Model")
print("-----")
print("Optimal k:", best_k)
print("MAE :", final_mae)
print("MSE :", final_mse)
print("RMSE:", final_rmse)
```

Result

The final k-NN regression model was evaluated using the optimal value of k obtained from the validation process. The resulting MAE, MSE and RMSE represent the performance of the selected model.

6.7 k Selection Algorithm

Algorithm: Selection of Optimal k

- Step 1:** Define a set of candidate values of k .
- Step 2:** Select the training and testing datasets.
- Step 3:** Standardize the input features.
- Step 4:** Select the first value of k .
- Step 5:** Calculate distances between test and training observations.
- Step 6:** Select the k nearest neighbours.
- Step 7:** Calculate the predicted target value.
- Step 8:** Calculate the RMSE for the selected k .
- Step 9:** Repeat Steps 4–8 for all candidate values of k .
- Step 10:** Compare the RMSE values.

Step 11: Select the k having the minimum RMSE.

Step 12: Use the selected k for the final k-NN model.

6.8 Summary

Different values of k were evaluated using a manual validation procedure. MAE, MSE and RMSE were calculated for each candidate value, and a validation curve was generated to visualize the effect of k on prediction error.

The value of k producing the lowest RMSE was selected as the optimal value and used for the final k-NN regression model.

(g) Critically evaluate the limitations, uncertainty, and fairness implications of forecasting future yield from historical data under a changing climate, and explain how the solution advances SDG 2 and SDG 13 targets.

7.1 Introduction

Locally Weighted Regression (LWR) is a non-parametric regression technique that gives greater importance to observations located closer to the query point.

Unlike ordinary regression, LWR does not attempt to learn one global relationship for the entire dataset. Instead, it fits a local model around each test observation. This makes LWR useful when the relationship between agricultural features and rice yield may vary across different observations.

In this project, LWR is implemented **from scratch using NumPy**, without using scikit-learn.

7.2 Working Principle of LWR

For a given test observation x , each training observation is assigned a weight according to its distance from x .

A Gaussian kernel is used to calculate the weights:

$$w_i = \exp\left(-\frac{d_i^2}{2\tau^2}\right)$$

where:

- w_i = weight of the i^{th} observation
- d_i = distance between the test observation and training observation
- τ = bandwidth parameter

A smaller distance produces a larger weight, while a larger distance produces a smaller weight.

The bandwidth τ controls the size of the local neighbourhood considered by the model.

7.3 Gaussian Weighting Kernel

The Gaussian kernel is implemented using the Euclidean distance between observations.

Code

```
def gaussian_weight(distance, tau):  
    return np.exp(  

```

```

        -(distance ** 2) / (2 * tau ** 2)
    )

print("Gaussian weighting kernel created successfully.")

```

7.4 Locally Weighted Regression Implementation

For each test observation, weights are calculated for all training observations. A weighted linear regression model is then fitted locally around the query point.

Code

```

def locally_weighted_regression(
    X_train,
    y_train,
    X_test,
    tau=1.0
):
    predictions = []

    # Add intercept column
    X_train_bias = np.c_[
        np.ones(len(X_train)),
        X_train
    ]

    for x in X_test:

        # Add intercept to test observation
        x_bias = np.insert(x, 0, 1)

        # Calculate distances

```

```

distances = np.sqrt(
    np.sum(
        (X_train - x) ** 2,
        axis=1
    )
)

# Calculate Gaussian weights
weights = np.exp(
    -(distances ** 2) /
    (2 * tau ** 2)
)

# Weight matrix
W = np.diag(weights)

# Weighted least squares
A = X_train_bias.T @ W @ X_train_bias
B = X_train_bias.T @ W @ y_train

theta = np.linalg.pinv(A) @ B

# Prediction
prediction = x_bias @ theta

predictions.append(prediction)

return np.array(predictions)

```



```
print("Locally Weighted Regression implemented successfully.")
```

7.5 LWR Prediction

An initial bandwidth value of $\tau = 1.0$ is selected. The model is then used to predict rice yield for the test observations.

Code

```
tau = 1.0

y_pred_lwr = locally_weighted_regression(
    X_train_scaled,
    y_train,
    X_test_scaled,
    tau
)

print("Bandwidth (tau):", tau)

print("\nFirst 10 actual values:")
print(y_test[:10])

print("\nFirst 10 LWR predictions:")
print(y_pred_lwr[:10])
```

7.6 Evaluation of LWR

The LWR predictions are evaluated using Mean Absolute Error, Mean Squared Error and Root Mean Squared Error.

Code

```
lwr_mae = np.mean(
    np.abs(y_test - y_pred_lwr)
)
```

```

lwr_mse = np.mean(
    (y_test - y_pred_lwr) ** 2
)

lwr_rmse = np.sqrt(lwr_mse)

print("Locally Weighted Regression Performance")
print("-----")
print("MAE :", lwr_mae)
print("MSE :", lwr_mse)
print("RMSE:", lwr_rmse)

```

Result

The Locally Weighted Regression model was successfully evaluated using MAE, MSE and RMSE. These values provide a basis for comparing LWR with the k-NN regression model.

7.7 Effect of Bandwidth

The bandwidth parameter τ controls the locality of the regression model.

A small bandwidth gives high importance to nearby observations and results in a more flexible model. A large bandwidth gives more observations significant weights and produces a smoother model.

Several bandwidth values are therefore tested.

Code

```

tau_values = [0.1, 0.3, 0.5, 1.0, 2.0]

lwr_results = []

for tau in tau_values:

    predictions = locally_weighted_regression(

```

```

X_train_scaled,
y_train,
X_test_scaled,
tau
)

mae_tau = np.mean(
    np.abs(y_test - predictions)
)

mse_tau = np.mean(
    (y_test - predictions) ** 2
)

rmse_tau = np.sqrt(mse_tau)

lwr_results.append([
    tau,
    mae_tau,
    mse_tau,
    rmse_tau
])

lwr_results = pd.DataFrame(
    lwr_results,
    columns=["Tau", "MAE", "MSE", "RMSE"]
)

print(lwr_results)

```

7.8 LWR Bandwidth Curve

The relationship between bandwidth and RMSE is visualized using a line graph.

Code

```
plt.figure(figsize=(8, 5))

plt.plot(
    lwr_results["Tau"],
    lwr_results["RMSE"],
    marker="o"
)

plt.xlabel("Bandwidth (Tau)")
plt.ylabel("RMSE")
plt.title("LWR Bandwidth Selection Curve")
plt.grid(True)

plt.show()
```

Interpretation

The bandwidth curve shows how prediction error changes as the locality parameter is varied. The bandwidth associated with the lowest RMSE provides the best performance among the tested values.

7.9 Selection of Optimal Bandwidth

The bandwidth producing the minimum RMSE is selected for the final LWR model.

Code

```
best_lwr_row = lwr_results.loc[
```

```

lwr_results["RMSE"].idxmin()
]

best_tau = float(best_lwr_row["Tau"])

print("Optimal bandwidth:", best_tau)
print("Minimum RMSE:", best_lwr_row["RMSE"])
print("Corresponding MAE:", best_lwr_row["MAE"])
print("Corresponding MSE:", best_lwr_row["MSE"])

```

Result

The optimal bandwidth was selected based on the minimum RMSE obtained during the bandwidth evaluation.

7.10 Final LWR Model

The final LWR model is evaluated using the selected optimal bandwidth.

Code

```

final_lwr_predictions = locally_weighted_regression(
    X_train_scaled,
    y_train,
    X_test_scaled,
    best_tau
)

final_lwr_mae = np.mean(
    np.abs(y_test - final_lwr_predictions)
)

final_lwr_mse = np.mean(
    (y_test - final_lwr_predictions) ** 2
)

```

)

```
final_lwr_rmse = np.sqrt(final_lwr_mse)
```

```
print("Final LWR Model")
```

```
print("-----")
```

```
print("Optimal Tau:", best_tau)
```

```
print("MAE :", final_lwr_mae)
```

```
print("MSE :", final_lwr_mse)
```

```
print("RMSE:", final_lwr_rmse)
```

7.11 Locally Weighted Regression Algorithm

Algorithm: Locally Weighted Regression

Step 1: Load the training and testing feature sets.

Step 2: Standardize the input features.

Step 3: Select a bandwidth parameter τ .

Step 4: Select one test observation as the query point.

Step 5: Calculate its distance from every training observation.

Step 6: Calculate Gaussian weights using the distances.

Step 7: Construct the weighted matrix.

Step 8: Calculate the locally weighted regression coefficients.

Step 9: Predict the target value for the query point.

Step 10: Repeat the process for all test observations.

Step 11: Calculate MAE, MSE and RMSE.

Step 12: Repeat the process for different bandwidth values and select the bandwidth with the lowest RMSE.

7.12 Summary

Locally Weighted Regression was implemented from first principles using NumPy. A Gaussian kernel was used to assign greater importance to observations closer to each query point.

Different bandwidth values were evaluated using RMSE, and the bandwidth with the lowest error was selected for the final LWR model. The resulting performance will be compared with the optimal k-NN model in the next section to study their prediction accuracy and bias-variance behaviour.